Module 1:

Topic 1: Structures

C program that represents a calendar for a week. Every day has:
dayName (eg., “Monday”)
tasks (array of string with maximum 3 tasks per day)
Note:
1. Define appropriate structures.
2. Allow the user to input tasks for any day.
3. Display all tasks grouped by the day

Code :

```c
#include<stdio.h>
#include<string.h>

struct Day{
    char day[10];
    char task[3][100];
};

int main(){
    struct Day week[7] = {
        {"Monday"},
        {"Tuesday"},
        {"Wednesday"},
        {"Thursday"},
        {"Friday"},
        {"Saturday"},
        {"Sunday"}
    };

    int d,t;

    while(1){
        printf("\n===== Days of the Week =====\n");
        for(int i=0;i<7;i++){
            printf("%d. %s\n",i+1,week[i].day);
        }

        printf("\nEnter day number (1-7) or 0 to Exit: ");
        scanf("%d", &d);
```

```
        getchar();

        if(d==0) break;
        if(d<1 || d>7){
            printf("Invalid day number!\n");
            continue;
        }

        printf("Enter number of tasks (Max 3): ");
        scanf("%d", &t);
        getchar();

        if(t>3) t=3;

        for(int i = 0; i < t; i++){
            printf("Enter Task %d: ", i + 1);
            fgets(week[d-1].task[i], 100, stdin);
            week[d-1].task[i][strcspn(week[d-1].task[i], "\n")] = '\0';
        }
    }

    printf("\n===== WEEKLY CALENDAR =====\n");
    for(int i = 0; i < 7; i++){
        printf("\n%s\n", week[i].day);
        for(int j = 0; j < 3; j++){
            if(strlen(week[i].task[j]) > 0){
                printf("- %s\n", week[i].task[j]);
            }
        }
    }
    return 0;
}
```

Output :

```
PS E:\college\embed\_Virtual training\Linux -assignments\Adv_C\module1> gcc q1.c -o q1.exe
PS E:\college\embed\_Virtual training\Linux -assignments\Adv_C\module1> ./q1.exe

===== Days of the Week =====
1. Monday
2. Tuesday
3. Wednesday
4. Thursday
5. Friday
6. Saturday
7. Sunday

Enter day number (1-7) or 0 to Exit: 2
Enter number of tasks (Max 3): 3
Enter Task 1: Jog 30 minutes
Enter Task 2: Learn DSA
Enter Task 3: Conquer the world

===== Days of the Week =====
1. Monday
2. Tuesday
3. Wednesday
4. Thursday
5. Friday
6. Saturday
7. Sunday

Enter day number (1-7) or 0 to Exit: 6
Enter number of tasks (Max 3): 1
Enter Task 1: Enjoy
```

```
===== Days of the Week =====
1. Monday
2. Tuesday
3. Wednesday
4. Thursday
5. Friday
6. Saturday
7. Sunday

Enter day number (1-7) or 0 to Exit: 0

===== WEEKLY CALENDAR =====

Monday

Tuesday
- Jog 30 minutes
- Learn DSA
- Conquer the world

Wednesday

Thursday

Friday

Saturday
- Enjoy

Sunday
```

Topic 2: Pointers
Write a function in C that takes a pointer to an integer array and its size, and then rearranges the array in place such that all even numbers arrear before odd numbers, preserving the original relative order using only pointer arithmetics (no indexing with [])

Code :

```c
#include <stdio.h>
#include <stdlib.h>

void rearrangeArray(int *arr, int size){
    for (int i=1;i<size;i++){
        if (*(arr+i)%2==0){
            int key=*(arr + i);
            int j=i-1;
            while (j >= 0 && *(arr+j)%2!= 0){
                *(arr+j+1)=*(arr+j);
                j--;
            }
            *(arr+j+1)=key;
        }
    }
}

int main(){
    int size;
    printf("Enter the size of the array: ");
    scanf("%d", &size);
    int *arr = (int *)malloc(size * sizeof(int));
    printf("Enter %d integers:\n", size);
    for (int i = 0; i < size; i++){
        printf("Element %d: ", i + 1);
        scanf("%d", arr + i);
    }
    rearrangeArray(arr, size);
    printf("\nRearranged array: ");
    for (int i = 0; i < size; i++){
        printf("%d ", *(arr + i));
    }
    free(arr);
    return 0;
}
```

Output :

```
PS E:\college\embed\_Virtual training\Linux -assignments\Adv_C\module1> gcc q2.c -o q2.exe
PS E:\college\embed\_Virtual training\Linux -assignments\Adv_C\module1> ./q2.exe
Enter the size of the array: 7
Enter 7 integers:
Element 1: 1
Element 2: 2
Element 3: 3
Element 4: 4
Element 5: 5
Element 6: 6
Element 7: 7

Rearranged array: 2 4 6 1 3 5 7
```

Topic 3: Arrays

You are given a 2D matrix of size n x n where each row and each column is sorted in increasing order.

Write a C function to determine whether the given key exists in the matrix using the most efficient approach.

Code :

```c
#include <stdio.h>
#include <stdlib.h>

int searchMatrix(int **mat,int rows,int cols,int key){
    int row = 0;
    int col = cols - 1;
    while(row<rows && col>=0){
        if (mat[row][col] == key) return 1;
        else if (mat[row][col] > key) col--;
        else row++;
    }
    return 0;
}

int main(){
    int rows, cols, key;
    printf("Enter number of rows and columns: ");
    scanf("%d %d", &rows, &cols);
    int **mat = (int **)malloc(rows * sizeof(int *));
    for (int i = 0; i < rows; i++){
        mat[i] = (int *)malloc(cols * sizeof(int));
```

```
        printf("Enter elements of row %d: ", i + 1);
        for (int j = 0; j < cols; j++) scanf("%d", &mat[i][j]);

    }
    printf("Enter key to search: ");
    scanf("%d", &key);
    if (searchMatrix(mat, rows, cols, key))  printf("Key found in the
matrix.\n");
    else  printf("Key not found in the matrix.\n");


    for (int i = 0; i < rows; i++) free(mat[i]);
    free(mat);
    return 0;
}
```

Output :

```
PS E:\college\embed\_Virtual training\Linux -assignments\Adv_C\module1> ./q3.exe
Enter number of rows and columns: 3 3
Enter elements of row 1: 1 2 3
Enter elements of row 2: 4 5 6
Enter elements of row 3: 7 8 9
Enter key to search: 5
Key found in the matrix.
PS E:\college\embed\_Virtual training\Linux -assignments\Adv_C\module1> ./q3.exe
Enter number of rows and columns: 2 2
Enter elements of row 1: 1 2
Enter elements of row 2: 3 4
Enter key to search: 0
Key not found in the matrix.
```